PlacementHub — College Placement Management Portal

Complete Project Architectural Breakdown & Interview Preparation Guide

Author: Candidate • **Tech Stack:** Python, Django 6.1, SQLite, HTML5, Vanilla CSS / Bootstrap 5 • **Pattern:** MTV

1. The 30-Second Elevator Pitch (For Interviewers)

"Tell me about your project:"
"PlacementHub is a full-stack web application designed to streamline the campus placement ecosystem by bridging students and corporate recruiters on a single, secure platform. Built on Django's robust architecture, it implements Role-Based Access Control (RBAC) separating student and recruiter workflows. Students can maintain an academic profile with resume uploads, search and filter active job opportunities by skillset, location, and CGPA, and track their application lifecycle. Recruiters can register multiple company entities, publish job openings with eligibility criteria, review applicants with one-click resume verification, and advance candidate statuses in real time. The system prevents duplicate applications using database-level constraints and guarantees academic eligibility compliance."

2. System Architecture & Tech Stack

Component	Technology / Implementation	Role & Key Advantage
Backend Framework	Python 3.13 / Django 6.1	Model-Template-View (MTV) pattern, built-in ORM, secure auth & CSRF protection.
Authentication & RBAC	Custom AbstractUser with role field	Enforces strict permissions between 'student' and 'recruiter' roles.
Database Layer	SQLite (Dev) / Relational Schema	Foreign keys with cascade deletion, unique multi-column constraints, index optimization.
Frontend UI/UX	Vanilla CSS Design Tokens + Bootstrap 5 Grid	Dark aesthetic (HSL tokens, glassmorphism, responsive cards, micro-animations, no bloat).
Static & Media Storage	Django Media Pipeline (resumes/)	Handles file uploads with validation, isolated storage, and served securely via MEDIA_URL.

3. Data Modeling & Database Schema (Core Entities)

The system is divided into modular Django apps following domain-driven design:

1. accounts.User (extends AbstractUser)

- Inherits standard Django security (username, hashed passwords, email, permissions).
  - Adds role: Choice field with 'student' and 'recruiter'.
  - Serves as the central identity model for all platform actors.

2. students.StudentProfile (OneToOne → accounts.User)

- Linked via models.OneToOneField(settings.AUTH_USER_MODEL, related_name='student_profile').
  - Fields: full_name, department, graduation_year, cgpa (DecimalField 4,2), skills (TextField), resume (FileField to resumes/).
  - Property skills_list: dynamically splits and trims comma-separated skills for frontend badges.

3. jobs.Company (ForeignKey → accounts.User)

- Represents corporate entities registered by recruiters (a recruiter can manage multiple companies).
  - Fields: recruiter (FK to User), name, description, website (URLField).

4. jobs.Job (ForeignKey → Company)

- Represents active or archived job openings.
  - Fields: company (FK), title, description, required_skills, minimum_cgpa (DecimalField), location, salary, deadline, is_active (BooleanField), created_at.
  - Property skills_list: parses skill strings into clean iterable badges.

5. applications.Application (ForeignKey → User, ForeignKey → Job)

- Connects student applicants to specific job postings.
  - Fields: student (FK to User), job (FK to Job), status (Choices: pending, shortlisted, selected, rejected), applied_at.
  - **Integrity Constraint:** models.UniqueConstraint(fields=['student', 'job'], name='unique_student_job_application') guarantees at the database level that no student can double-apply.

4. End-to-End User Workflows

Student Workflow	Recruiter Workflow
1. Registration & Auth: • Registers with username, email, password, and role='student'. • Auto-redirects to Student Dashboard. **2. Profile & Resume Setup:** • Enters academic branch, grad year, CGPA, and key skills. • Uploads verified PDF resume with inline status check. **3. Job Discovery & Exploration:** • Accesses 'Browse Jobs' portal. • Filters openings by keyword query (title, skills, company), location, and CGPA limit. **4. Application & Eligibility Verification:** • System verifies student CGPA ≥ Job minimum CGPA. • Visual indicator updates to 'Already Applied' button once submitted. **5. Real-Time Tracking:** • Monitors application status (Pending → Shortlisted → Selected) on 'My Applications' dashboard.	**1. Registration & Auth:** • Registers with role='recruiter'. • Directs to Recruiter Executive Command Center. **2. Company Management:** • Adds multiple company profiles with logo, website, and description. **3. Job Creation & Publishing:** • Creates job listings specifying salary package, location, required skills, and minimum CGPA criteria. • Can toggle listings between Active and Inactive on the fly. **4. Candidate Review Pipeline:** • Reviews applicants filtered by job opening or review status. • Inspects candidate department, CGPA, and key skills. • One-click preview of candidate's uploaded PDF resume. **5. Decision Pipeline Management:** • Updates candidate status with zero scroll/filter loss (Shortlist, Select, Reject).

5. Technical Challenges & Architectural Decisions

• Database-Level Concurrency & Integrity:

Instead of relying on application-level checks that are vulnerable to race conditions, I implemented models.UniqueConstraint(fields=['student', 'job']). Combined with get_or_create() in the view, this guarantees idempotency and zero duplicate applications.

• Complex Multi-Criteria Dynamic Filtering:

Implemented dynamic query building in browse_jobs and recruiter_applications using Django's Q objects. This allows simultaneous searching across multiple attributes (job title, description, skills, company name, candidate email, and academic branch) while chaining status and eligibility criteria.

• Query Optimization with select_related:

To prevent the classic 'N+1 query problem', all application queries utilize select_related('student', 'student__student_profile', 'job', 'job__company'). This executes an efficient SQL JOIN in a single query rather than hitting the database for every rendered row.

- **Preserving UI State on Mutation:**

When recruiters update application statuses from filtered tab views, the request carries a next parameter referencing request.get_full_path. The view redirects back to the referring state, eliminating context loss during bulk candidate evaluation.

- **Defensive Template Rendering:**

Created safe model properties (skills_list) rather than ad-hoc template filters, ensuring clean separation of concerns and preventing template rendering crashes when profiles or skill sets are partially filled.

6. Top Technical Interview Q&A; (Rehearsal Sheet)

Q1: Why did you choose Django for this project instead of Flask or FastAPI?

Answer: Django was the optimal choice because a placement portal requires robust authentication, administrative controls, an ORM for relational modeling, and built-in security out of the box. Django's 'batteries-included' philosophy provides CSRF protection, SQL injection prevention, session management, and form validation natively, enabling rapid development without having to assemble multiple third-party libraries.

Q2: How is Role-Based Access Control (RBAC) enforced across views?

Answer: I extended AbstractUser with a role field ('student' vs 'recruiter'). In the view layer, access is protected using a combination of the @login_required decorator and role-validation guards (e.g., if request.user.role != 'recruiter': return redirect('home')). On the template layer, conditional blocks dynamically adjust navigation menus and action buttons based on user.role.

Q3: How do you prevent a student from applying to the same job twice?

Answer: I solved this at both the database layer and the presentation layer. At the database layer, Application.Meta includes a models.UniqueConstraint(fields=['student', 'job']). At the view layer, get_or_create() handles attempts gracefully. In the UI, the template checks if job.id exists in the student's applied_job_ids set, swapping the 'Apply' button for an inactive 'Already Applied' badge.

Q4: How did you prevent the N+1 database query problem in the applications list?

Answer: If a recruiter views 50 applications, fetching application.student, application.student.student_profile, application.job, and application.job.company individually would trigger over 200 SQL queries. I solved this by invoking .select_related('student', 'student__student_profile', 'job', 'job__company'), which performs SQL JOIN operations in one single database query.

Q5: How are file uploads (resumes) managed securely?

Answer: Resumes are handled through Django's FileField(upload_to='resumes/'). The model enforces an isolated subdirectory inside MEDIA_ROOT. Uploads are accepted only via POST with enctype='multipart/form-data' and validated through StudentProfileForm with .pdf constraints to prevent arbitrary script execution.

Q6: How would you scale this application to support 50,000+ students and 500+ companies?

Answer: In production, I would make several architectural enhancements:
1. **Database:** Migrate from SQLite to PostgreSQL with read replicas and connection pooling (PgBouncer).
2. **Media Storage:** Offload resume file storage from the web server to AWS S3 or Google Cloud Storage with pre-signed secure URLs.
3. **Background Tasks:** Use Celery with Redis for asynchronous background tasks such as sending notification emails and parsing resumes.
4. **Caching:** Implement Redis caching for high-read pages like job listings.
5. **Pagination:** Implement server-side pagination with Paginator to limit payload sizes on large job/application lists.

7. Project Directory & Route Cheat Sheet

App / Module	Route / URL Pattern	Target View & Role

core	/	home — Landing page for all visitors
accounts	/accounts/register/ /accounts/login/ /accounts/logout/	User registration, session login, logout
accounts	/accounts/student/dashboard/	Student metrics command center
accounts	/accounts/recruiter/dashboard/	Recruiter company & job stats dashboard
students	/students/profile/	Student profile & resume management (Student)
jobs	/jobs/browse/	Browse & search active jobs with filters (Student)
jobs	/jobs/<id>/apply/	One-click apply with CGPA eligibility guard (Student)
jobs	/jobs/my-applications/	Track status of applied jobs (Student)
jobs	/jobs/companies/ /jobs/companies/create/	Company list & registration (Recruiter)
jobs	/jobs/jobs/my-jobs/ /jobs/jobs/create/	Job postings management & job creation (Recruiter)
jobs	/jobs/jobs/<id>/toggle/	Activate / Deactivate job posting (Recruiter)
jobs	/jobs/applications/	Filter candidates & resume review (Recruiter)
jobs	/jobs/applications/<id>/update/	Advance candidate status in pipeline (Recruiter)